

AutoSolver Agent

系统设计方案

面向外卖配送任务分配问题的自主求解智能体

Helga Celia Jenry

Ustiantseva Diana

Zhang Yuqi

1 项目概述

1.1 任务背景

在外卖配送系统中,如何高效地把大量配送订单分配给可用骑手,是决定运营成败的关键环节。真实场景需要同时权衡配送时间、配送距离、骑手接单意愿,以及合单配送(把多个订单合并交给同一骑手)的可能性。

系统还具有三个重要特性:其一,允许拒绝订单,被拒订单计入惩罚;其二,对骑手接起对应订单的概率(意愿)有预测;其三,允许把同一订单同时指派给多位骑手,最先接起订单的骑手获得该订单。因此,系统需要在多个目标之间取得平衡:最大化接单订单数量、最大化系统收益(以分数表示),并通过智能合单提升资源利用率。

1.2 问题定义

给定一组配送订单(任务)与可用骑手,以及每个「任务-骑手」组合的预计算分数,需要求出一个最优分配方案,使接单订单数量最大化,同时总分数最小。该问题是一个带覆盖约束与合单结构的组合优化问题,属于经典指派/集合覆盖问题的扩展。

1.3 设计目标

本方案围绕赛题给出的四项核心交付能力构建一个 AutoSolver Agent 智能体系系统:

1. 自主策略探索:Agent 能自主提出并尝试不同求解策略(贪心、ILP、启发式、LLM 直接推理等),无需人工指定使用哪种算法。
2. 自动评估与筛选:每次尝试后自动计算目标分数,判断是否优于当前最优解,并据此决定保留或丢弃。
3. 迭代改进循环:根据历史实验结果自主调整下一轮策略(例如发现贪心在合单场景表现差,则自动转向其他方向)。
4. 输出最终解:在 10 秒/测试用例的时间限制内,输出当前最优分配方案。

2 问题建模

2.1 输入与输出

输入:制表符分隔(TSV)的候选指派记录,每条记录包含四个字段:

字段	含义
task_id_list	任务编号列表;单任务为一个编号,合单为两个任务编号
courier_id	骑手编号
total_score	该「任务-骑手」组合的预计算分数(成本相关)
willingness	骑手接起该订单的预测意愿(概率)

输出:一组 (task_str, [courier, ...]) 元组构成的列表。每个任务对应的骑手列表可包含一个主派骑手与若干备份骑手——把同一任务派给多名骑手以降低无人接单的风险,由最先接起者获得订单。

2.2 优化目标(字典序)

目标采用字典序(lexicographic)优先级,先后顺序不可互换:

1. 第一优先:最大化被覆盖(成功指派)的任务数量。
2. 第二优先:在覆盖数相同的前提下,最小化总成本/总分数。该成本公式由真实提交结果标定得到。

说明:系统统一以「越低越好」的方式最小化目标分数;每个未覆盖任务计入固定惩罚 100。若评测端采用「越高越好」口径,则在评测适配层取负转换。

2.3 关键决策与约束

- 合单(2-task bundles):一名骑手同时承接两个任务,可显著降低单位成本,但会占用运力、提高失败连带风险。
- 备份骑手:同一任务派发给主派 + 备份,提升接单概率,但会增加占用与潜在成本,需要在覆盖率与成本间权衡。
- 拒单惩罚:每个未覆盖任务固定惩罚 100,促使方案尽量提高覆盖。
- 时间约束:每个测试用例必须在 10 秒内完成求解并产出可行解。
- 可行性约束:骑手运力不被超额占用、任务编号合法、合单结构正确;非法解在评分前被直接拒绝。

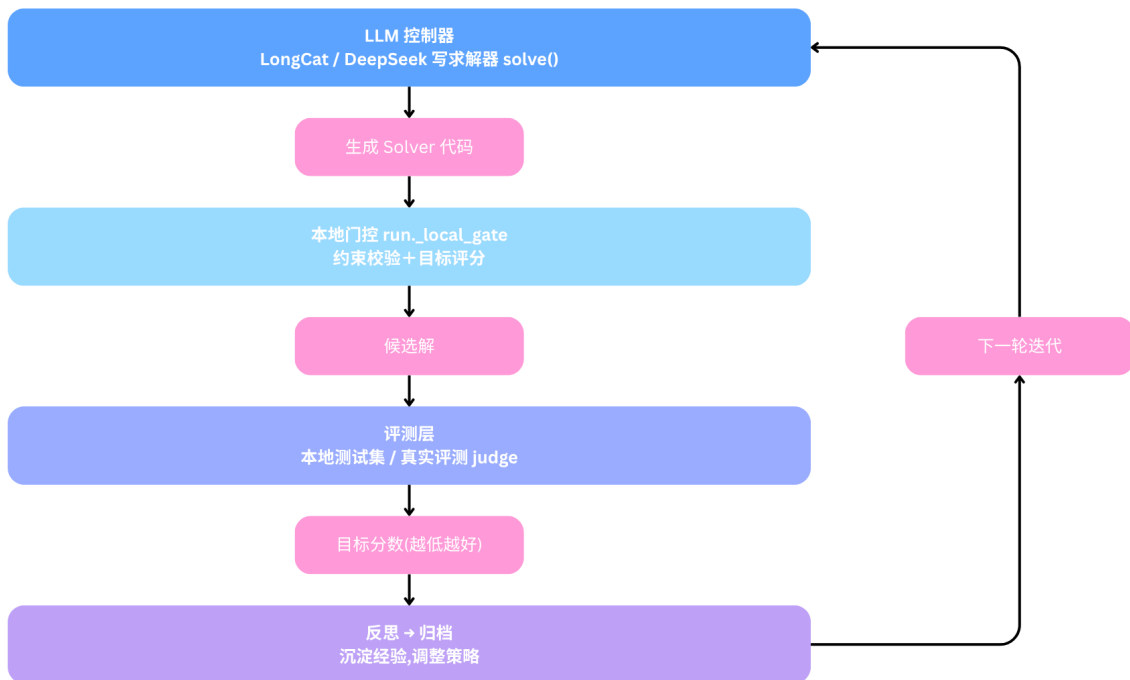
3 系统总体架构

3.1 设计理念

系统的核心不是某一个固定的求解算法,而是一个「会写求解器、会评估、会反思、会改写」的智能体闭环。大语言模型(LLM)负责生成与修订 `solve(input_text)` 求解函数;一个确定性的本地门控负责校验与评分;评测层提供本地与真实两种反馈;记忆与归档模块沉淀跨轮、跨运行的经验,驱动策略持续进化。

3.2 核心闭环

一轮迭代的数据流如下,分数越低代表方案越优:



本地门控会在 `dataset/` 下的每个 `.txt`(真实样例 `large_seed301.txt` 与合成样例 `syn_*.txt`)上运行求解器,先校验输出合法性,再在已标定时依据 `cost_model.json` 评分。求解器代码在带硬超时的子进程中运行,非法解在评分前被拒绝。

3.3 模块划分

系统由三层组成:智能体控制层(策略生成与决策)、求解与评测层(执行与打分)、记忆与校准层(经验沉淀与口径对齐)。各模块及其职责见下表。

文件 / 模块	职责
agent.py	主循环:生成 → 门控 → 提交 → 反思,串联整个迭代闭
judge_adapter.py	评测适配:封装 submit_and_score() 与本地门控,是适配不同评测端时唯一需要改动的文件
archive.py	跨运行记忆:维护 run_log.jsonl 与 solver_archive/,沉淀历史解与得分
calibrate.py	从真实评测响应反推成本公式 $\text{cost}(\text{total_score}, \text{willingness})$ 与未覆盖惩罚,生成 cost_model.json
reconcile.py	拟合本地预测与真实分数的线性映射 $\text{real} \approx a \cdot \text{pred} + b$,产出 recon.json
memory.py	把历史经验蒸馏为精简的 memory_brief.md,作为
make_synthetic.py	生成 10 种典型样例 dataset/syn_*.txt 用于离线开发
calibrate_synthetic.py	调校合成样例的意愿值,使其逼近冠军解的真实逐样例
dashboard.py / watch.py	从日志构建静态 dashboard.html;watch.py 提供实时
best_solver.py	迄今找到的最优求解器(自动生成)
standalone_solver.py	人工编写的 ILP / 贪心求解器(独立于 Agent,作对照基

过程中生成的产物(均在 .gitignore 中):run_log.jsonl、calls.jsonl、agent_state.json、cost_model.json、recon.json、solver_archive/、dashboard.html、memory_brief.md。

4 核心模块设计

4.1 主循环 agent.py

agent.py 是系统的中枢,按「生成 → 门控 → (可选)提交 → 反思」的节奏推进。每一轮:从记忆中读取精简经验与当前最优解作为上下文,提示 LLM 产出 best-of-N 个候选求解器;对每个候选先过本地门控做可行性校验与评分,保留本轮最优;在 submit 模式下再按预算决定是否提交真实评测;最后把结果反思后写回归档。每轮都会裁剪 LLM 上下文,长期经验由记忆模块承载,避免上下文膨胀。

4.2 评测适配 judge_adapter.py

该文件把「如何评分」这一外部依赖收敛到单一入口 `submit_and_score()`,从而让主循环与具体评测端解耦。它同时实现了离线本地门控。系统支持四种评测模式:

JUDGE_MODE	说明
local	离线本地评分(默认),在本地样例上打分
http	调用 <code>hackathon.mykeeta.com</code> 评测 API:登录 → POST / judge → 轮询 /result/{job_id}
browser	无头 Playwright 浏览器自动化(需配置 CSS 选择器)
manual	Agent 写出 <code>current_solver.py</code> ,人工把真实分数回填

4.3 记忆与归档

`archive.py` 提供跨运行的长期记忆:`run_log.jsonl` 记录每轮的策略、得分与决策,`solver_archive/` 保存历史求解器源码。`memory.py` 进一步把冗长历史蒸馏为 `memory_brief.md`——一份高密度的经验摘要,作为 LLM 每轮的长期记忆注入。两者配合,使 Agent 即便在上下文受限时也能「记住」哪些方向有效、哪些已被证伪。

4.4 校准三件套

离线分数若与真实评测口径不一致,迭代方向就会被误导。系统用三个相互配合的模块对齐口径:

1. `calibrate.py`:从真实评测响应中挖掘 `case_results[].detail`,学习成本函数 `cost(total_score, willingness)` 与未覆盖任务惩罚,产出 `cost_model.json`。
2. `reconcile.py`:用「本地预测 / 真实均值」样本对拟合线性修正 $real \approx a \cdot pred + b$,使 submit 模式的门控以真实单位筛选候选,产出 `recon.json`。
3. `calibrate_synthetic.py`:调校合成样例中的意愿值,使其逐样例得分逼近冠军真实分数,提升离线套件的代表性。

未校准时,本地套件仅是近似;一旦完成一次或多次真实提交,Agent 即改用标定后的模型做 go/no-go 决策。

5 Agent 工作机制

本节逐项说明系统如何实现赛题要求的四项核心能力。

5.1 自主策略探索

LLM 控制器在 react 模式下自行决定每轮采取的动作与求解思路,可在以下策略族中自由组合、切换,无需人工指定算法:贪心分配、整数线性规划(ILP)、启发式/局部搜索、以及 LLM 直接推理。每轮可一次性生成 best-of-N 个候选(candidates),扩大搜索面。

5.2 自动评估与筛选

每个候选求解器在本地门控中自动运行并打分:先做合法性校验(运力、任务合法性、合单结构)非法解直接拒绝;合法解依据(已标定的)成本模型计算目标分数。系统按字典序(覆盖优先、成本次之)与当前最优解比较,自动决定保留或丢弃,无需人工介入。

5.3 迭代改进循环

反思环节把本轮结果(策略、得分、失败原因)写入归档,并由记忆模块蒸馏为经验摘要回注下一轮。Agent 据此自主调整方向——例如当历史显示贪心在合单密集场景下得分偏高时,会主动转向 ILP 或带合单偏好的启发式。整个过程形成「实验—评估—反思—调整」的自驱闭环。

5.4 输出最终解

求解器在带硬超时的子进程中执行,确保单个测试用例严格控制在 10 秒内;超时或异常的候选被判为非法并丢弃,不影响已保留的最优解。系统始终维护 best_solver.py 作为当前最优方案,可随时输出可行的最终分配结果。

6 求解策略库

Agent 可生成并组合下列策略。它们在覆盖率、成本、稳定性与求解时间上各有取舍,由迭代过程自动择优。

策略	思路	适用 / 取舍
贪心	按单位收益或意愿排序逐步指派	快、稳;合单与全局最优场景下易陷入局部解
整数线性规划 (ILP)	用 pulp / ortools 建模覆盖与合单约束求最优	解质量高;规模大时需控制求解时间以满足 10s
启发式 / 局部搜索	在可行解上做合单重组、备份增删等邻域搜索	在质量与时间间折中,适合大规模实例
LLM 直接推理	由模型直接对小规模实例给出指派	灵活、可融合先验;需门控兜底校验合法性
合单 / 备份组合	在上述基础上叠加 2-task 合单与备份骑手	提升覆盖与降本的关键杠杆,需平衡风险

7 评测与校准体系

7.1 本地门控与真实评测

本地门控用于离线快速迭代,在合成套件与真实样例上即时打分;真实评测对接 hackathon.mykeeta.com,以登录、提交、轮询结果的流程获取权威分数。两者形成「离线高频探索 + 真实低频验证」的双层反馈。

7.2 提交预算管理

真实评测每队每天限 20 次提交(北京时间零点重置)。Agent 在 `agent_state.json` 中追踪剩余预算,并据此调整探测激进程度;在 http 模式首次运行时会自动花费 1 次提交完成成本公式标定(—calibrate)。

7.3 口径对齐

通过第 4.4 节的校准三件套,把本地预测分数映射到真实口径,确保「本地认为更优」与「真实更优」高度一致,从而让有限的真实提交次数用在真正有把握的候选上。

8 运行模式与监控

8.1 运行模式

模式	说明
react	LLM 控制器自主选择动作(默认),探索性最强
linear	固定的 生成 → 评分 → 反思 流程,行为可预期
submit	在每日预算内审慎进行真实提交

8.2 常用参数

参数	作用
--iterations N	迭代步数(react / linear)
--candidates N	每轮生成的 best-of-N 候选数
--provider longcat deepseek	选择 LLM 提供方
--model NAME	模型名(默认 LongCat-2.0-Preview / deepseek- chat-v3-turbo)
--max-submits N	submit 模式下的真实提交上限
--calibrate	用 1 次评测探测生成 cost_model.json 后退出
--consolidate	把历史蒸馏为 memory_brief.md 后退出

8.3 监控

watch.py 提供实时终端面板,可在第二个终端中观察运行状态;运行结束后由 dashboard.py 从日志构建静态 dashboard.html,用于离线复盘策略演化与分数曲线。

9 技术选型

层次	技术	用途
智能体 / LLM	LongCat-2.0-Preview(默认)、DeepSeek-chat	生成与修订求解器、策略决策与反思

求解器	Python 3.10+, 可选 pulp / ortools	在生成代码内部建模并求解优化问题
运行环境	标准库为主的 Agent 主循环 + 子进程隔离	安全执行不可信生成代码, 带硬超时
评测对接	HTTP API, Playwright(浏览器	对接真实评测端, 获取权威
数据与记忆	JSONL 日志、JSON 模型文件、Markdown 摘要	跨运行记忆、成本模型与经验沉淀

10 关键设计决策与权衡

- 评测适配单点收敛: 把评测差异全部封装进 judge_adapter.py, 使核心闭环与评测端彻底解耦, 迁移到新评测端只改一处。
- 本地高频 + 真实低频: 用离线门控承担绝大多数试错, 真实提交仅用于校准与终验, 既快又省提交预算。
- 子进程 + 硬超时: 生成代码在隔离子进程中执行并强制超时, 兼顾安全性与 10s 时限, 异常解自动剔除。
- 记忆蒸馏控制上下文: 每轮裁剪上下文、长期经验落盘, 既保留历史洞见又控制 token 成本。
- 校准驱动决策: 用标定后的成本模型与线性修正对齐口径, 避免离线分数误导迭代方向。

11 风险与应对

风险	应对
生成代码非法或超时	子进程硬超时 + 评分前合法性校验, 非法解直接丢
离线分数与真实口径偏差	校准三件套对齐口径, 真实提交后切换标定模型决
每日提交预算耗尽	agent_state.json 跟踪预算, 按剩余次数调节探测激
LLM 上下文膨胀 / 漂移	每轮裁剪上下文, 经验由 memory_brief.md 长期承
合成样例不具代表性	calibrate_synthetic.py 用真实逐样例分数反向调

12 总结

AutoSolver Agent 把外卖配送任务分配问题抽象为「自主策略探索—自动评估筛选—迭代改进—输出最终解」的智能体闭环,完整覆盖了赛题的四项交付能力。系统以 LLM 生成并迭代求解器、以确定性门控保证可行与评分、以校准体系对齐真实口径、以记忆与归档沉淀经验,在 10 秒/用例、每日 20 次提交的约束下,持续将目标分数推向更优,实现覆盖率最大化与成本最小化的字典序目标。